



UNIVERSIDAD
DE GRANADA



Cátedra de Inteligencia Artificial

Universidad de Granada – Unicaja

Agente Conversacional Bancario

Integrantes

Pablo Martín Palomino `pablomarpa@correo.ugr.es`

Pablo Reyes Sousa `sousareyp@correo.ugr.es`

Natalia Yue Gallardo Pretel `nataliayue@correo.ugr.es`

Septiembre de 2026

Índice

1	Introducción y arquitectura general	3
1.1	Objetivos y enfoque de la solución	3
1.2	Arquitectura técnica	3
2	Interacción Natural y Experiencia de Usuario (UX)	4
2.1	Integración de voz y texto	4
2.2	Generación de respuestas en streaming	4
2.3	Interfaz adaptativa y accesible	6
3	Configuración, Generación y Seguridad de la Base de Datos	6
3.1	Generación de un histórico realista	6
3.2	Defensa en profundidad y aislamiento del LLM	7
4	Motor de Inteligencia Artificial	7
4.1	Gestión del historial y prevención del desbordamiento de contexto	7
4.2	System Prompt dinámico y reglas de comportamiento	8
4.3	Uso de Tool Calling	8
5	Operaciones Bancarias Seguras	9
5.1	Atajos de latencia	9
5.2	Flujo transaccional de Bizum	10
6	Consultas y Analítica Avanzada	11
6.1	Traducción del lenguaje natural a SQL para el histórico de movimientos . .	11
6.2	Enfoque híbrido: heurística y estadística	12
6.2.1	Detección de pagos recurrentes y suscripciones	12
6.2.2	Proyecciones de gasto y analítica predictiva	13
7	Motor Visual Generativo	14
7.1	Generative UI y separación de responsabilidades	14
7.2	Gráficos Vega-Lite y tablas dinámicas sin plantillas	15
8	Evaluación y resultados	15
8.1	Por qué se construyó un banco de pruebas	15
8.2	Qué se mide y por qué se separa	15
8.3	Resultados	16
8.4	Decisiones de diseño tomadas a partir de la medición	17
8.4.1	La decisión de mostrar un gráfico	17
8.4.2	El tamaño del <i>system prompt</i>	17
8.5	Validación de los componentes deterministas	18
8.5.1	Detección de pagos recurrentes	18
8.5.2	Proyección de gasto	19
8.5.3	Flujo de operaciones	19
8.6	Limitaciones conocidas	19
8.6.1	Alcance funcional	20
8.6.2	Límites técnicos y de operación	20
8.6.3	Límites de la demostración en materia de seguridad	21

8.6.4	Límites de la propia evaluación	21
9	Conclusiones y Cumplimiento del Baremo	21
9.1	Conclusiones	21
9.2	Cumplimiento del baremo técnico del reto	22
10	Líneas de trabajo futuras	23
10.1	Mejoras sobre lo ya construido	23
10.2	Mejoras que requieren rediseño	24
11	Reparto del trabajo	24
11.1	Pablo Martín Palomino	24
11.2	Natalia Yue Gallardo Pretel	25
11.3	Pablo Reyes Sousa	26
11.4	Uso de asistentes de IA	26

1. Introducción y arquitectura general

1.1. Objetivos y enfoque de la solución

El proyecto “Habla con tu dinero” es nuestra solución al reto propuesto por la Cátedra de Inteligencia Artificial de la Universidad de Granada junto a Unicaja.

Con él, buscamos que el usuario pueda realizar consultas y operaciones financieras de manera sencilla e interactiva.

Para garantizar un funcionamiento fiable, se ha separado la interpretación de las consultas de la ejecución de las operaciones. Mientras que el modelo de lenguaje facilita la interacción con el usuario, el *backend* se encarga de obtener los datos, realizar los cálculos y ejecutar las operaciones correspondientes.

1.2. Arquitectura técnica

La aplicación se divide en tres bloques principales: la interfaz de usuario, el *backend* y la base de datos.

- **Interfaz de usuario (Frontend):** Se ha desarrollado utilizando HTML y JavaScript. La comunicación con el servidor se realiza mediante WebSockets, manteniendo una conexión abierta durante la interacción. Esto permite recibir las respuestas de forma progresiva y actualizar la interfaz dinámicamente. Además, se pueden mostrar elementos como gráficos, tablas y mensajes dentro de la propia conversación.
- **Motor principal (Backend):** Se ha desarrollado en Python utilizando FastAPI. Se encarga de recibir las peticiones del usuario, gestionar el contexto de la conversación y coordinar los diferentes componentes del sistema. También permite resolver directamente determinadas consultas sin utilizar el modelo de lenguaje. Por ejemplo, una consulta sobre el saldo de una cuenta puede resolverse directamente desde el backend, reduciendo el tiempo de respuesta.
- **Base de datos y seguridad:** La información de la aplicación se almacena en una base de datos SQLite. El acceso a los datos se gestiona desde el backend, que determina qué operaciones puede realizar cada parte del sistema. Las consultas que únicamente necesitan información utilizan acceso de lectura, mientras que las operaciones que modifican datos o implican movimientos de dinero son ejecutadas directamente por la lógica de la aplicación.

Como se muestra en la Figura 1, el *backend* se estructura de forma modular. Un controlador principal (`main.py`) gestiona las conexiones y delega el flujo en un agente (`agent.py`). Este agente utiliza un catálogo de herramientas (`tools.py`) para derivar la ejecución hacia el motor visual (`graficos.py`), la lógica analítica (`analitica.py`), las APIs bancarias (`banking_api.py`) o la base de datos (`database.py`).

Asimismo, el modelo de lenguaje (por defecto, Qwen3) se ejecuta de forma local y privada a través de Ollama, comunicándose exclusivamente con el *backend* sin acceso directo a los datos.

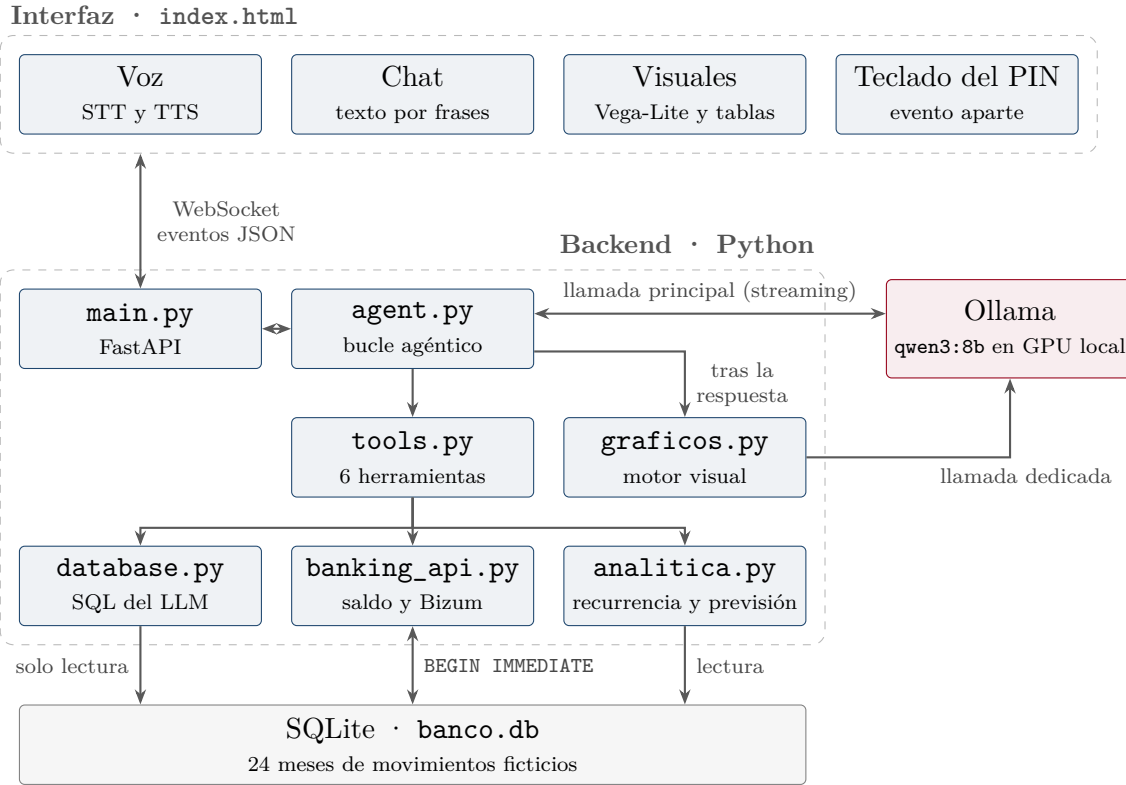


Figura 1: Arquitectura del sistema.

2. Interacción Natural y Experiencia de Usuario (UX)

2.1. Integración de voz y texto

El reconocimiento de voz se apoya en la API estándar del navegador (*Speech Recognition*). El micrófono se activa pulsando un botón en la interfaz o manteniendo presionada la barra espaciadora. Para dar *feedback* inmediato al usuario y evitar la sensación de latencia, se utilizan los resultados intermedios (*interim results*); de este modo, el texto va apareciendo en la pantalla a medida que se habla, enviándose al servidor justo cuando el usuario termina la locución.

Para la lectura de las respuestas se utiliza la API de síntesis de voz (*Speech Synthesis*). No se espera a que el modelo genere todo el párrafo, sino que la emisión de audio se sincroniza con el flujo de datos (*streaming*) y se va reproduciendo por frases completas, utilizando los signos de puntuación como delimitadores. Esto soluciona problemas frecuentes de lectura con cifras o decimales y evita cortes a mitad de palabra. Además, el usuario puede personalizar el tono (*pitch*), la velocidad (*rate*) y la voz del asistente desde un panel de configuración, guardándose sus preferencias localmente en el navegador (*localStorage*).

2.2. Generación de respuestas en streaming

Toda la comunicación en tiempo real entre el *cliente* y el *servidor* se realiza a través de una conexión WebSocket abierta de forma continua. Gracias a este canal, el usuario no tiene que esperar a que el servidor procese la respuesta completa para empezar a leerla,

sino que el texto va apareciendo en la interfaz frase a frase, a medida que el modelo las completa. La unidad no es el *token* sino la frase, porque es lo que la síntesis de voz puede leer sin cortar palabras ni tropezar con los decimales.

Además del texto de la respuesta, pueden transmitirse actualizaciones relacionadas con los datos mostrados en pantalla, como cambios en el saldo o instrucciones para representar tablas y gráficos.

Esta estrategia reduce el tiempo de espera percibido y permite que la interfaz comience a mostrar información mientras el procesamiento de la consulta continúa en segundo plano. De esta forma, la generación de la respuesta y la actualización de los distintos elementos de la interfaz pueden realizarse de forma progresiva.

La Figura 2 recoge la secuencia completa de un turno en el que el usuario hace una consulta sobre su histórico. El texto llega frase a frase y la síntesis de voz empieza con la primera; la tabla o el gráfico se generan en un tramo posterior, cuando la respuesta ya está completa.

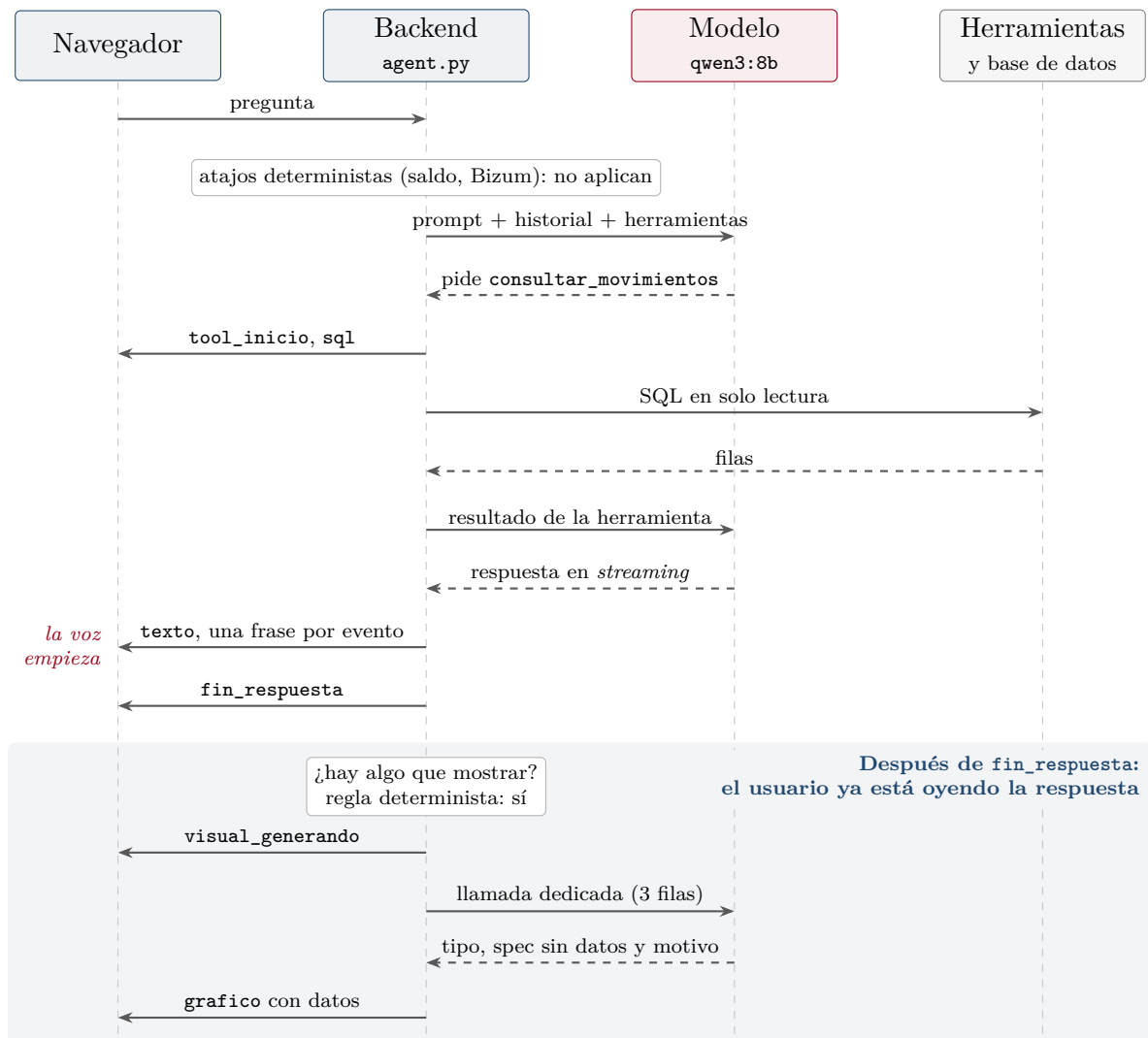


Figura 2: Secuencia de un turno con consulta al histórico y gráfico.

2.3. Interfaz adaptativa y accesible

La interfaz no se limita a mostrar texto plano. Cuando una consulta devuelve datos comparables, el sistema lo detecta y enriquece la respuesta con gráficos o tablas interactivas.

El diseño original delegaba en el modelo de lenguaje la decisión de cuándo pintar estos elementos visuales (controlado mediante la variable `VISUALES_AL_LLM`). Sin embargo, las métricas revelaron que darle esta responsabilidad al LLM bloqueaba la emisión de la voz: el modelo solía detenerse a generar la compleja estructura de datos del gráfico antes de empezar a redactar el texto en español, lo que disparaba la latencia inicial hasta unos 12,4 segundos de media.

Para resolverlo, la configuración actual oculta por defecto las herramientas visuales al modelo principal. Ahora es el *backend* quien evalúa de forma determinista el volumen y tipo de los datos obtenidos en la consulta para decidir si estos son graficables. Si lo son, lanza una llamada dedicada al LLM para construir la especificación del gráfico, pero lo hace estrictamente después de emitir el evento `fin_respuesta`. Dado que la memoria de la tarjeta gráfica empleada no permite paralelizar dos peticiones al LLM, esta segunda solicitud visual debe esperar su turno en cola obligatoriamente.

La ventaja obtenida es que el usuario escucha la respuesta hablada de inmediato (el tiempo hasta la voz se redujo a unos 6,7 segundos) y, mientras atiende a la explicación del asistente, la interfaz muestra un indicador de carga hasta que el gráfico o la tabla aparecen en pantalla.

3. Configuración, Generación y Seguridad de la Base de Datos

3.1. Generación de un histórico realista

Para evaluar el asistente en escenarios financieros verosímiles, necesitamos un conjunto de datos suficientemente rico. Para ello, el proyecto incluye un módulo de generación sintética, `seed.py`, que crea un histórico de operaciones correspondiente a un periodo de 24 meses.

El algoritmo de generación es completamente determinista, inicializado con `random.seed(42)`. Esto facilita tanto el desarrollo como la repetición de las pruebas, ya que los resultados se pueden comparar sobre un conjunto de datos conocido.

Los datos generados simulan patrones de consumo realistas que exigen distintos niveles de complejidad analítica por parte del asistente:

- **Gastos recurrentes y variables:** se combinan cargos fijos y regulares (como el alquiler, la luz o el agua) con transacciones con alta variabilidad (compras en supermercados, repostaje en gasolineras, restaurantes, etc.).
- **Pagos con periodicidad anual:** se incluyen operaciones separadas por periodos de tiempo más amplios, como la renovación anual del seguro del coche. Esto permite comprobar el funcionamiento de las consultas que requieren aplicar filtros sobre periodos de tiempo extensos.

- **Cambios en los gastos:** se simulan subidas de cuota a mitad del histórico, como incrementos documentados en el precio de plataformas de *streaming* para comprobar si el sistema es capaz de reconocer dicho incremento.

De esta forma, la base de datos proporciona un conjunto de pruebas variado que permite comprobar tanto consultas sencillas como aquellas que requieren analizar información de diferentes periodos.

3.2. Defensa en profundidad y aislamiento del LLM

Una parte importante del sistema es controlar el acceso del modelo de lenguaje a la base de datos. El modelo puede generar consultas para obtener información a partir de las peticiones realizadas por el usuario, por lo que es necesario evitar que dichas consultas puedan modificar los datos almacenados.

Para ello, el acceso a la base de datos, implementado en `database.py`, incorpora varias medidas de protección independientes:

- **Acceso de solo lectura:** cuando el modelo necesita consultar información del historial, el backend utiliza una conexión SQLite en modo solo lectura mediante `mode=ro`. De esta forma, las operaciones que intenten modificar la base de datos, como `DROP` o `UPDATE`, son rechazadas por el propio motor de SQLite.
- **Validación de las consultas:** antes de ejecutar una consulta generada por el modelo, el sistema comprueba su contenido mediante una serie de expresiones regulares. Se bloquean palabras clave asociadas a operaciones de modificación de datos o de la estructura de la base de datos, como `INSERT`, `DELETE` o `ALTER`. Además, se comprueba que la consulta comience por `SELECT` o `WITH`.
- **Limitación del número de resultados:** las consultas que devuelven un número elevado de registros pueden generar una cantidad de información innecesaria para el modelo. Para evitarlo, el sistema establece un límite de 50 filas mediante la constante `MAX_FILAS = 50`. Cuando una consulta devuelve más resultados, estos se truncan antes de incorporarse al contexto de la conversación.

Estas medidas permiten separar la generación de las consultas de su ejecución. El modelo de lenguaje se encarga de construir la consulta a partir de la petición del usuario, mientras que el backend controla qué consultas pueden ejecutarse y con qué permisos se accede a la base de datos.

4. Motor de Inteligencia Artificial

4.1. Gestión del historial y prevención del desbordamiento de contexto

Los modelos de lenguaje tienen un límite en la cantidad de texto que pueden recordar a la vez (lo que se conoce como ventana de contexto, que en nuestro caso es de 8192 tokens). Como las instrucciones iniciales y la explicación de las herramientas ya ocupan bastante espacio, es necesario controlar cuánto historial de la conversación le enviamos para no saturarlo.

Para solucionar esto, la clase **Agente** (en `agent.py`) incluye una función llamada `recortar_historial` que limita la memoria a los últimos 24 mensajes. Este recorte no se hace de cualquier manera, sino que el sistema siempre conserva el primer mensaje (el *System Prompt*) y busca el inicio de una intervención del usuario (`role: user`) para cortar a partir de ahí.

De esta forma, nos aseguramos de no partir por la mitad una interacción entre el modelo y una herramienta. Si cortáramos mal, el modelo podría recibir la respuesta de una herramienta sin saber por qué la había pedido, lo que provocaría un error al intentar generar el siguiente mensaje

4.2. System Prompt dinámico y reglas de comportamiento

El comportamiento del asistente y las reglas que debe seguir se definen en un *System Prompt* que construimos en el archivo `config.py`.

Este texto no es estático, sino que se genera cada vez que arranca la aplicación para poder incluir información actualizada. Por ejemplo, le pasamos la fecha de hoy usando `date.today().isoformat()` para que el modelo entienda a qué nos referimos cuando le preguntamos por los gastos de este mes o la semana pasada. También le adjuntamos la estructura de la base de datos (`ESQUEMA_BD`) para que sepa qué tablas y columnas existen cuando tenga que crear consultas SQL.

Además, en este *prompt* le damos instrucciones claras sobre cómo debe expresarse. Le pedimos que trate al usuario de tú, que formatee el dinero al estilo español (por ejemplo, "1.234,56 €") y le prohibimos usar formato Markdown como negritas o cursivas. Esto último es muy importante para que el sistema que lee el texto en voz alta lo haga de forma natural, sin tropezar leyendo símbolos.

4.3. Uso de Tool Calling

Para que el modelo pueda interactuar con la aplicación, utilizamos la funcionalidad de *Tool Calling*. En el archivo `tools.py` definimos en formato JSON una lista con todo lo que el asistente puede invocar: consultar datos, preparar un envío de Bizum...

Para ejecutar el modelo de lenguaje en local, garantizar la privacidad de los datos y no depender de suscripciones o servicios en la nube hemos utilizado Ollama.

El problema que tenemos es que las herramientas en `tools.py` las están definidas usando la estructura de Anthropic, pero Ollama no soporta directamente este formato (esto pues empezamos el proyecto pensando que usaríamos API de Claude pero finalmente decidimos priorizar el uso de un modelo local).

Para solucionarlo, en `agent.py` está la función `adaptar_herramientas_a_openai`, que hace de puente y traduce automáticamente nuestras herramientas al formato correcto antes de enviárselas al modelo, permitiendo que la conexión funcione sin problemas.

El proceso completo de interacción funciona como se ilustra en la Figura 3: el usuario hace una pregunta, y el modelo evalúa si necesita usar alguna herramienta para responder. Si es así, interrumpe su respuesta y devuelve un mensaje especial pidiendo ejecutarla. Nuestro *backend* recibe esa petición, ejecuta el código en Python (por ejemplo, hace la consulta a la base de datos) y le devuelve el resultado al modelo añadiendo un mensaje con el rol `tool`. Finalmente, el modelo lee ese resultado y ya dispone de los datos concretos para redactar la respuesta que escuchará el usuario.

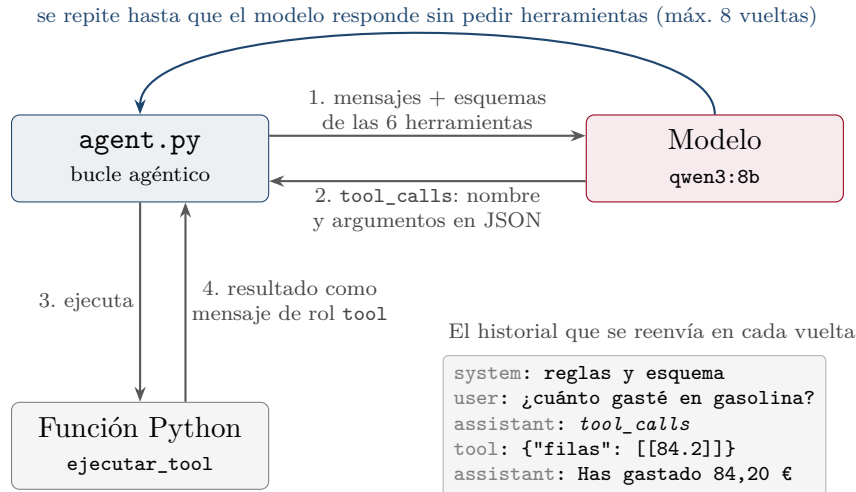


Figura 3: Ciclo de *tool calling*. El modelo nunca ejecuta nada: devuelve el nombre de la herramienta y sus argumentos, el *backend* ejecuta la función en Python y le devuelve el resultado como un mensaje más del historial. Con ese dato ya concreto, el modelo redacta la respuesta final.

5. Operaciones Bancarias Seguras

5.1. Atajos de latencia

Durante las pruebas, nos dimos cuenta de que no siempre hace falta recurrir al modelo de lenguaje. Por ejemplo, cuando el usuario simplemente pregunta "¿cuál es mi saldo?", pasar la pregunta por el LLM añadía unos segundos de latencia innecesaria para devolver un dato directo.

Para optimizar esto, en el archivo `agent.py` incluimos la función `es_consulta_saldo`. Si el mensaje coincide con esta validación, el *backend* llama directamente a la API `consultar_saldo` y responde al momento, saltándose por completo al modelo.

El filtro funciona detectando palabras clave (como saldo o dinero disponible) y comprobando que el resto de la frase solo contenga términos de relleno autorizados en la lista `_RELLENO_SALDO`. Hemos sido muy restrictivos a propósito con estas palabras permitidas, para evitar que consultas que requieran información adicional sean identificadas incorrectamente como una consulta de saldo.

En este caso, los errores tienen distinto coste:

- **Falso negativo:** si una consulta de saldo simple no pasa el filtro, la petición llegará al LLM, que usará la herramienta correspondiente y responderá correctamente, solo que tardará un poco más.
- **Falso positivo:** si confundimos una pregunta compleja con una simple, el sistema fallará. Por ejemplo, si el usuario pregunta "¿cuál era mi saldo el mes pasado?", el atajo le devolvería el saldo actual, dando una respuesta completamente incorrecta.

Por este motivo, ante la más mínima duda (si hay palabras fuera de la lista de relleno), el filtro prefiere no aplicar el atajo y dejar que el modelo analice la intención completa de la frase de forma segura.

5.2. Flujo transaccional de Bizum

Mover dinero es la operación más crítica de la aplicación. Por seguridad, la ejecución de un Bizum nunca se delega libremente al modelo de lenguaje; el LLM o las expresiones regulares solo ayudan a entender la intención, pero la ejecución está controlada de principio a fin por el *backend*. El proceso, ilustrado en la Figura 4, sigue estas etapas:

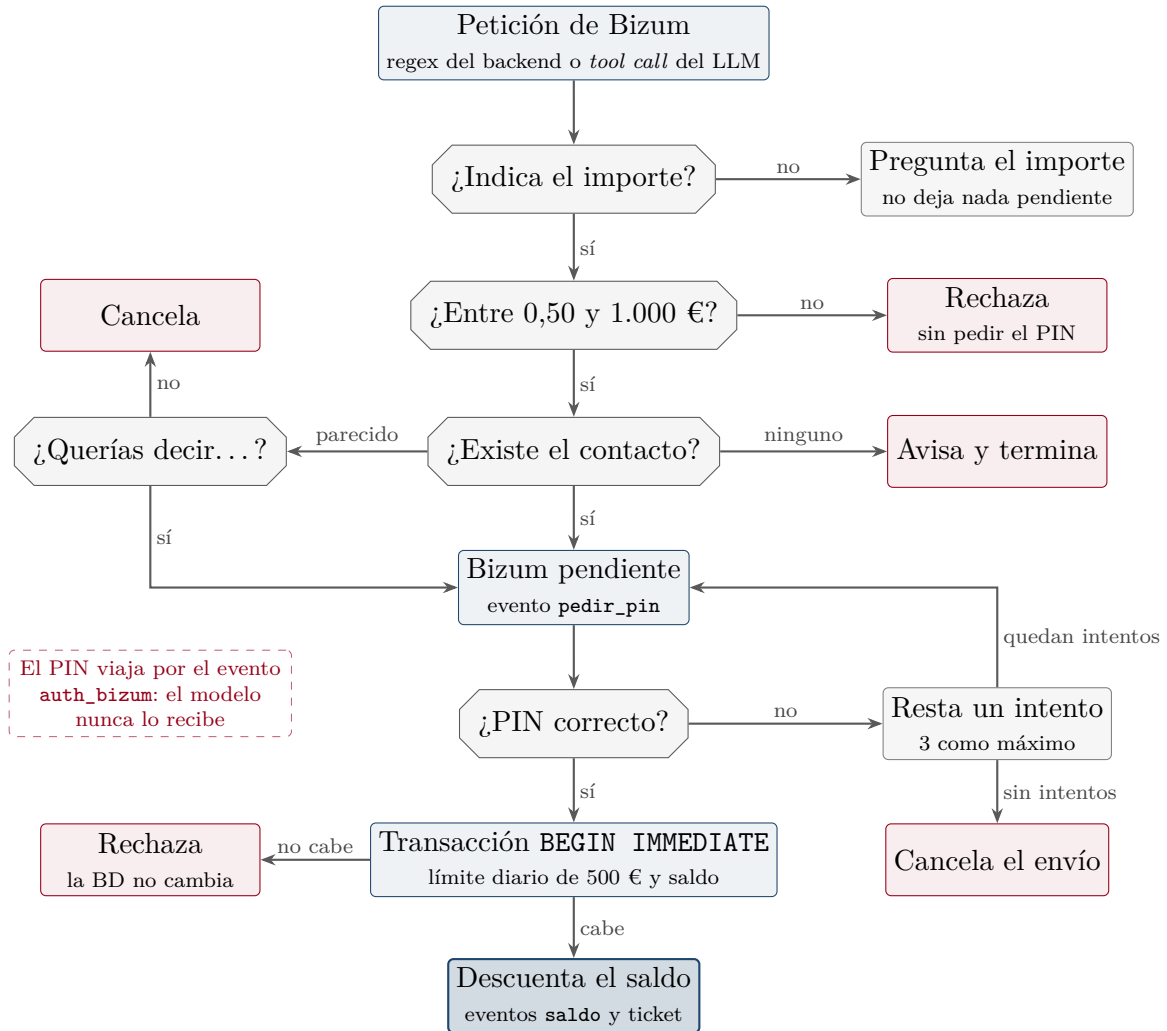


Figura 4: Flujo de un envío de Bizum. Todas las comprobaciones se hacen en el backend y en este orden: el importe se valida antes de pedir el PIN, y el límite diario y el saldo se comprueban dentro de la misma transacción que los descuenta.

1. **Extracción y validación inicial:** la función `extraer_petition_bizum` identifica en la petición del usuario los datos necesarios para realizar el envío, como el destinatario, el importe y el concepto. La extracción permite reconocer diferentes formas de expresar una misma operación. Antes de continuar con el proceso, el importe se comprueba para verificar que se encuentra dentro de los límites establecidos, entre 0,50€ y 1000€.
2. **Resolución del contacto:** una vez obtenido el destinatario, llamamos a la función `buscar_contacto_bizum`, que comprueba los contactos disponibles. En caso de que

el nombre proporcionado no coincida exactamente, se utilizan coincidencias parciales mediante `get_close_matches` para identificar posibles contactos. Si existe una coincidencia, esta se muestra al usuario antes de continuar con la operación.

3. **Autenticación desde el frontend:** cuando los datos de la operación son válidos, el estado de la conversación pasa a “Bizum pendiente” y el *backend* envía mediante WebSocket la señal `pedir_pin`. El *frontend* muestra entonces un teclado numérico para introducir el PIN. De esta forma, el PIN se introduce fuera del campo de texto de la conversación y no forma parte del historial que recibe el modelo de lenguaje.
4. **Validación y ejecución de la operación:** una vez validado el PIN, el *backend* ejecuta la función `api_enviar_bizum`. La modificación del saldo se realiza dentro de una transacción SQLite iniciada mediante `BEGIN IMMEDIATE`, utilizando `transaccion_escritura`. Esta instrucción establece el bloqueo necesario para evitar que dos operaciones de escritura se ejecuten simultáneamente sobre los mismos datos. De esta forma, la comprobación del saldo y su posterior actualización se realizan dentro de la misma transacción.

Este diseño mantiene separadas la interpretación de la petición y la ejecución de la operación bancaria. El modelo de lenguaje puede identificar la intención del usuario y proporcionar los datos necesarios, pero las validaciones y la modificación de los datos se realizan mediante la lógica del backend.

6. Consultas y Analítica Avanzada

6.1. Traducción del lenguaje natural a SQL para el histórico de movimientos

Para que el usuario pueda preguntar por sus gastos pasados de forma natural, utilizamos *Text-to-SQL* a través de la herramienta `consultar_movimientos`. De esta forma, dejamos que el modelo traduzca la pregunta directamente a una consulta en SQLite.

Para que el modelo no se equivoque interpretando los datos, le pasamos unas reglas muy claras en el *System Prompt*:

- **Tratamiento de signos:** En nuestra base de datos, los gastos se guardan en negativo. Para sumar lo gastado y que la cifra no confunda al usuario, le explicamos al modelo que tiene que usar `SUM(-importe)` y mostrar siempre el resultado en positivo.
- **Gestión temporal:** Le indicamos cómo usar funciones nativas como `strftime` (ya que el manejo de fechas en SQLite puede generar errores si no se unifican los formatos).
- **Autocorrección:** Si el modelo genera un SQL inválido que produce un error en SQLite, el sistema no se rompe. La capa de base de datos intercepta el error y se lo devuelve al agente en el siguiente turno. De esta forma, el modelo se da cuenta de su propio fallo, corrige la sentencia y lo vuelve a intentar automáticamente. El bucle no puede prolongarse indefinidamente: el agente tiene un tope de `MAX_ITERACIONES_AGENTE = 8` vueltas por turno, que cuenta todas las llamadas a herramientas, incluidas las correcciones.

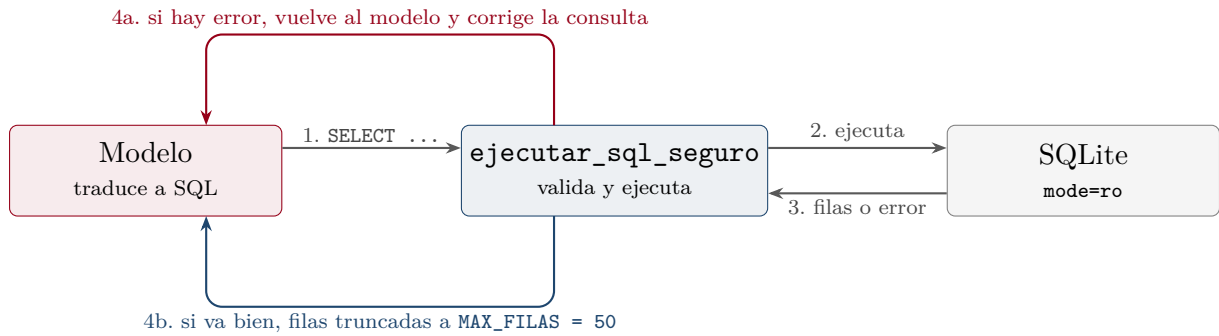


Figura 5: Traducción a SQL y autocorrección. La consulta que escribe el modelo nunca llega directamente a la base de datos: pasa por una capa que comprueba que sea una única sentencia de lectura y la ejecuta sobre una conexión `mode=ro`. Si SQLite devuelve un error, ese error se le entrega al modelo como resultado de la herramienta y corrige en la vuelta siguiente, dentro del tope de ocho iteraciones por turno.

Con este diseño, el modelo actúa como un traductor ágil entre el usuario y los datos, mientras que la ejecución segura y la gestión de fallos siguen controladas por el *backend*.

6.2. Enfoque híbrido: heurística y estadística

Hay preguntas que un modelo de lenguaje pequeño no sabe deducir solo escribiendo SQL. Averiguar qué pagos son periódicos o predecir cuánto dinero nos va a sobrar a final de mes exige hacer cálculos sobre todo el historial y buscar patrones estadísticos.

Por eso, apostamos por un enfoque híbrido: el análisis estadístico está programada de forma determinista en Python dentro del módulo `analitica.py`, y el modelo de lenguaje solo entra en juego al final, para entender qué ha pedido el usuario y redactarle el resultado.

6.2.1. Detección de pagos recurrentes y suscripciones

En la base de datos no hay ninguna columna que nos diga si un pago es una suscripción mensual o un recibo fijo; es algo que hay que inferir analizando el histórico.

Para detectarlos, nuestro algoritmo no se fija en que el importe sea siempre idéntico porque recibos como la luz o el agua varían cada mes, sino en la regularidad de los días que pasan entre un cobro y otro.

El procedimiento es el siguiente: calculamos la mediana de los intervalos entre los pagos de un mismo comercio y medimos cuánto se desvían de ella. Si ese factor de irregularidad es menor o igual a 0,06 (valor elegido empíricamente), y la mediana encaja en una cadencia conocida (mensual, trimestral, anual, etc.) con un margen de tolerancia del 20 %, confirmamos que es un pago recurrente.

La Figura 6 compara los dos casos sobre los datos reales del histórico: Netflix se cobra cada 30 o 31 días, mientras que en una gasolinera se reposta cuando toca. El hueco entre ambos grupos es amplio, y el umbral cae dentro de él.

Esto nos permite identificar desde la cuota de Netflix hasta el seguro del coche que llega una vez al año. Además, al revisar todo el histórico, este mismo algoritmo detecta automáticamente si alguna de estas suscripciones ha subido de precio recientemente.

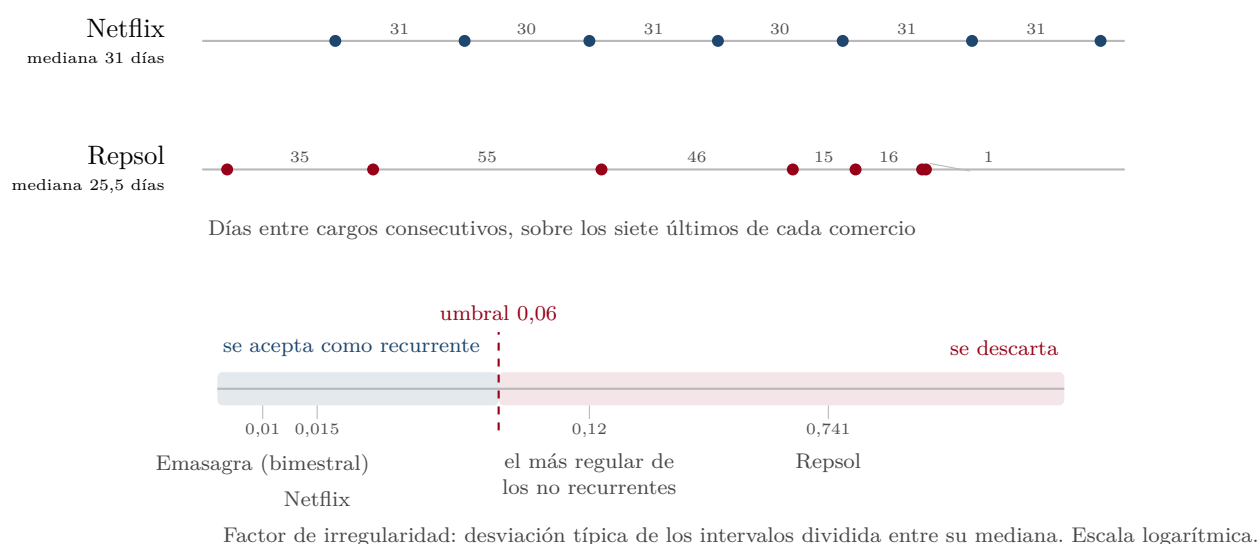


Figura 6: Cómo se distingue un pago recurrente. Lo que separa una suscripción de un comercio que se visita a menudo no es el importe, sino la regularidad de los intervalos entre cargos. Los valores medidos sobre el histórico generado dejan un margen amplio entre ambos grupos, y el umbral de 0,06 cae dentro de ese hueco.

6.2.2. Proyecciones de gasto y analítica predictiva

Para estimar el gasto total de un mes no resulta suficiente con extrapolar directamente el gasto acumulado hasta el momento. Esta aproximación puede producir resultados poco representativos al comienzo del mes, especialmente cuando existen pagos fijos que todavía no se han realizado.

Para que la previsión sea realmente útil, decidimos dividirla en dos bloques:

- **Gastos fijos:** Como ya hemos detectado los pagos recurrentes en el paso anterior, sabemos cuánto cuestan al mes. Los sumamos enteros a la proyección directamente, independientemente de si ya se han cobrado este mes o están pendientes. Hacerles una regla de tres rompería la estimación.
- **Gasto variable:** Aquí sí hacemos una estimación, mezclando el ritmo de gasto de este mes con la media histórica de los últimos 12 meses. El peso de cada parte se ajusta dinámicamente según el día en el que estemos: el día 3 pesa más la historia, y el día 25 nos fijamos más del ritmo actual

Para afinar y validar este modelo usamos *backtesting* sobre 23 meses de histórico real, lo que nos permitió medir exactamente cuánto nos equivocábamos cada día del mes según la categoría de gasto

Los resultados del análisis se resumen antes de enviarlos al modelo de lenguaje. En lugar de proporcionarle únicamente los valores numéricos calculados, el backend puede generar conclusiones como “fiabilidad alta” o “tendencia por encima de la media”. De esta forma, el modelo recibe información ya procesada y se encarga principalmente de presentarla al usuario.

7. Motor Visual Generativo

7.1. Generative UI y separación de responsabilidades

Para enriquecer las respuestas de texto, implementamos un motor de *Generative UI* que construye gráficos y tablas sobre la marcha. En las primeras pruebas, le pedíamos al modelo que generase el componente visual completo (tanto la estructura como los datos). Sin embargo, esto presentaba dos problemas graves: disparaba el consumo de *tokens* (retrasando enormemente la respuesta) y provocaba alucinaciones, ya que el modelo a veces se equivocaba o inventaba cifras al transcribirlas.

Para solucionarlo, en el archivo `graficos.py` separamos el diseño visual de los datos reales:

- **Decisión de viabilidad:** después de ejecutar una consulta SQL, el backend analiza el resultado para determinar si puede representarse mediante un componente visual. Por ejemplo, se puede considerar representable un resultado con varias filas y columnas numéricas, o una fila que contenga varias métricas que puedan compararse.
- **Diseño de la visualización:** cuando el resultado es apto para su representación, se realiza una llamada independiente al modelo de lenguaje. En esta llamada se proporcionan los nombres de las columnas y una muestra reducida de los datos. El modelo utiliza esta información junto con el contexto de la consulta para determinar si resulta más apropiado utilizar una tabla o un gráfico y generar la estructura correspondiente. La respuesta incluye también el campo `razonamiento`, que indica el motivo de la elección.
- **Inyección de los datos:** el modelo no genera los valores que se muestran finalmente al usuario. Devuelve únicamente la estructura del componente visual. El backend incorpora posteriormente los datos reales obtenidos de la base de datos antes de enviar el componente al frontend mediante WebSocket.

Esta separación, que resume la Figura 7, permite que el modelo se encargue de determinar cómo representar la información, mientras que los datos mostrados proceden directamente de la consulta realizada por el backend.

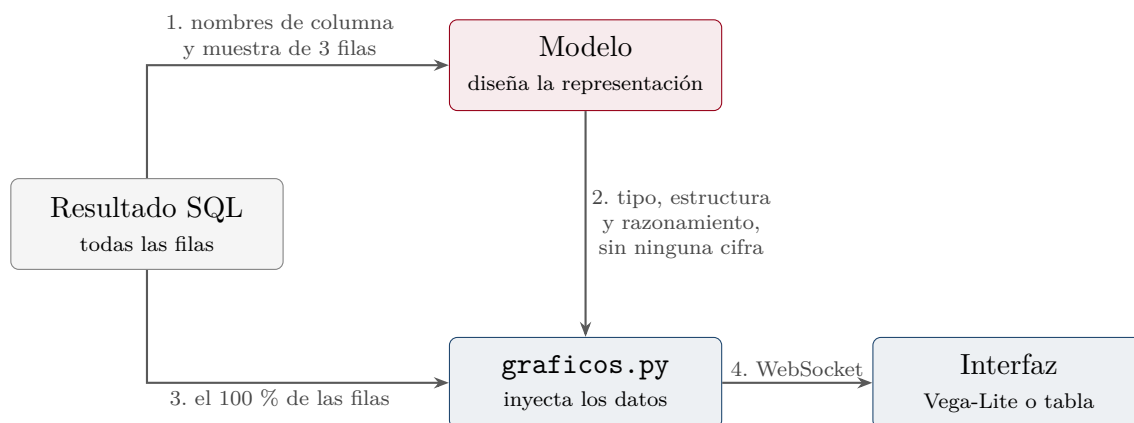


Figura 7: Motor visual generativo. El modelo solo decide cómo representar la información; los valores los pone el *backend*, que ya los tiene. Esto ahorra unos 700 *tokens* de generación por elemento visual y elimina de raíz que el modelo se invente cifras al copiarlas.

7.2. Gráficos Vega-Lite y tablas dinámicas sin plantillas

El sistema no utiliza una colección fija de plantillas para representar los resultados. En su lugar, el modelo de lenguaje genera la estructura del componente a partir del tipo de información obtenida y de la consulta realizada.

- **Gráficos con Vega-Lite v5:** cuando los datos se pueden representar de forma más adecuada mediante una visualización, el modelo genera una especificación JSON compatible con Vega-Lite. En ella determina el tipo de marca utilizado mediante **mark**, como líneas, barras o áreas, y establece la correspondencia entre las columnas y los diferentes elementos del gráfico mediante **encoding**. También puede definir los títulos y etiquetas que se muestran en la interfaz.
- **Tablas dinámicas:** cuando resulta más importante consultar valores concretos o cuando los resultados contienen varios atributos, el modelo puede seleccionar una representación mediante tabla. En este caso, determina qué columnas deben mostrarse y en qué orden, además de asignar reglas de formato mediante etiquetas como “euros”, “fecha” o “texto”. El frontend utiliza esta información para representar cada columna con el formato correspondiente.

De esta forma, la representación visual se genera a partir de los resultados concretos de cada consulta, sin necesidad de definir previamente una plantilla específica para cada tipo de pregunta.

8. Evaluación y resultados

8.1. Por qué se construyó un banco de pruebas

A diferencia de un error de programación, que se manifiesta con una excepción, los errores producidos por el modelo de lenguaje son silenciosos: el sistema puede generar una respuesta errónea sin que se detecte de inmediato.

Durante el desarrollo se detectó un caso que ilustra el problema. Ante la pregunta “¿he gastado más este año que el año pasado?”, el modelo generaba una consulta con `strftime('%Y-%m')` en lugar de `strftime('%Y')`. La consulta comparaba el mes actual contra el mes anterior, pero la respuesta los presentaba como años. El asistente respondía que se había gastado más este año cuando la realidad era 17.870 € frente a 26.643 €: la conclusión estaba invertida y se expresaba con seguridad. Revisando la interfaz, la respuesta resultaba convincente.

Por este motivo se desarrolló `tests/evaluar.py`, un banco de preguntas que se ejecuta contra el agente real y mide su comportamiento de forma sistemática.

8.2. Qué se mide y por qué se separa

El banco contiene 44 preguntas etiquetadas, distribuidas entre los tipos de consulta que menciona el enunciado (gastos, comparativas y suscripciones) y otros añadidos durante el desarrollo: seguimiento multiturno, operaciones de Bizum, saldo, ingresos, preguntas ambiguas, límites del asistente y previsión.

Se puntúan cuatro indicadores por separado, porque responden a fallos de naturaleza distinta:

- **Elección de herramienta.** Si el usuario pregunta por sus suscripciones y el modelo decide escribir una consulta SQL manual en lugar de invocar `analizar_suscripciones`, se considera un fallo aunque la cifra final resulte ser correcta. Este indicador se evalúa mediante un espía sobre la función `ejecutar_tool`, permitiendo registrar también las peticiones que el *backend* resuelve de forma directa (como el saldo) y que no dejan rastro en el historial del modelo.
- **Respuesta final.** Que la consulta sea correcta no garantiza que el modelo traslade bien la cifra al texto. Es lo único que oye el usuario, y se comprueba que los valores esperados aparezcan con formato español.
- **Refuerzo visual.** Se puntúa en los dos sentidos: no mostrar nada cuando hay varios valores comparables es un fallo, y mostrar algo cuando la respuesta es un único dato también. Una tabla cuenta igual que un gráfico, porque la elección entre ambos corresponde al modelo y las dos son respuestas válidas.
- **Validez de la especificación.** Que llegue un gráfico no significa que se vea. Una especificación sin el campo `mark` supera la validación del backend y falla después en el renderizado, sin dejar rastro. Se puntúa por separado del indicador anterior porque son fallos de cosas distintas: uno mide al modelo decidiendo y el otro al modelo redactando.

Adicionalmente, medimos un quinto indicador: ejecutamos la consulta SQL generada por el agente en paralelo a una consulta SQL escrita a mano (la referencia), y comparamos los resultados. No se puntúa porque el modelo puede interpretar la pregunta de forma diferente pero válida. Además, nunca comparamos el texto en crudo de la consulta SQL, ya que existen multitud de sentencias válidas para obtener un mismo resultado.

Por último, evaluamos la latencia en dos tramos. El tramo más relevante mide el tiempo transcurrido hasta emitir el evento `fin_respuesta`, que marca el momento en el que el texto de la respuesta está completo. Esta métrica representa una cota superior de la espera percibida por el usuario, ya que la síntesis de voz arranca antes (con la primera frase).

8.3. Resultados

Las métricas se obtuvieron realizando tres repeticiones independientes de las 44 preguntas, utilizando el modelo local `qwen3:8b` y regenerando la base de datos sintética antes de cada evaluación.

Indicador	Resultado
Elección de herramienta	96,8 %
Respuesta final correcta	90,5 %
Refuerzo visual cuando corresponde	96,3 %
Especificaciones válidas	100 %
Latencia mediana hasta la voz	3,4 s

Cuadro 1: Resultados sobre 44 preguntas y 3 repeticiones.

8.4. Decisiones de diseño tomadas a partir de la medición

El banco de pruebas no se utilizó únicamente para verificar el resultado final. En varios casos, la medición contradijo una decisión previa y motivó un cambio de arquitectura.

8.4.1. La decisión de mostrar un gráfico

Inicialmente, la decisión de acompañar una respuesta con un gráfico se dejaba al modelo mediante instrucciones en el *system prompt*. La medición mostró que esa decisión no era estable: variaba ante cualquier modificación del *prompt*, incluso ante cambios que no trataban sobre gráficos.

Pregunta	v1	v2	v3	v4
Ranking de comercios	0/3	2/3	0/3	0/3
Comparación mensual	0/3	0/3	0/3	0/3
Comparación por categorías	0/3	2/3	2/3	0/3
Serie de seis meses	0/3	3/3	3/3	0/3
Comparación interanual	0/3	0/3	0/3	0/3
Evolución mensual	3/3	3/3	3/3	3/3

Cuadro 2: Veces que cada pregunta produce un gráfico, sobre cuatro versiones del *system prompt*. Entre la tercera y la cuarta solo cambiaron dos líneas relativas al tratamiento de fechas.

A partir de este resultado se reorganizó la responsabilidad. La decisión de *si* existe algo que representar pasó al backend, mediante una regla determinista sobre la forma del resultado: varias filas con alguna columna numérica, o una sola fila con dos o más cifras comparables. La decisión de *qué* representación utilizar (tabla o gráfico), con qué marca, qué ejes y con qué justificación, permanece en el modelo, en una llamada dedicada de tarea única.

El indicador pasó del 16,7 % al 100 % en la medición que validó el cambio, sin variación en la precisión de las respuestas ni en la latencia percibida. En la tabla de resultados actual figura un 96,3 %: la diferencia no viene de la regla determinista, sino de una pregunta en la que el modelo elige la herramienta equivocada y su resultado sí es tabulable. La Figura 8 recoge ese cambio junto al de la latencia, que es la otra decisión que se tomó a partir de una medición.

8.4.2. El tamaño del *system prompt*

Durante el desarrollo, el *system prompt* fue creciendo al incorporar nuevas reglas. La medición mostró que ese crecimiento degradaba reglas anteriores. La causa era la duplicación: las mismas seis herramientas se describían tanto en el *prompt* como en sus esquemas JSON, y ambos textos se envían en cada petición. Se eliminó la parte duplicada (la que indica cuándo llamar a cada herramienta, que el esquema ya expresa y en mejor posición, junto a la definición de la función) y se conservó la que el esquema no puede expresar: cómo interpretar el resultado devuelto.

Se verificó una por una que ninguna de las diecisiete reglas obtenidas mediante medición se hubiera perdido en el proceso. Se recuperaron dos preguntas, una de ellas registrada desde el inicio como limitación asumida del modelo: no lo era.

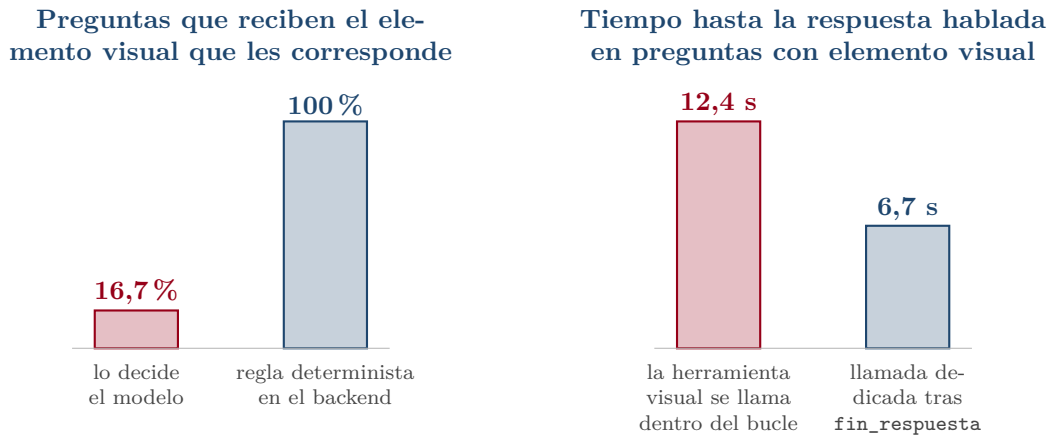


Figura 8: Las dos decisiones que más cambiaron al medirlas. A la izquierda, mover al *backend* la decisión de *si* mostrar algo; a la derecha, sacar la generación del elemento visual fuera del camino crítico. La segunda medición corresponde solo a las preguntas que acaban generando una tabla o un gráfico, no a la mediana de las 44.

	Antes	Después
Contexto fijo por petición	3.822 tokens	3.208 tokens
Porcentaje de la ventana	47 %	39 %
Elección de herramienta	97,6 %	97,6 %
Respuesta final correcta	85,7 %	90,5 %

Cuadro 3: Efecto de eliminar las instrucciones duplicadas.

8.5. Validación de los componentes deterministas

Los componentes que no dependen del modelo se validan por separado, con pruebas que no requieren el LLM y que por tanto son deterministas y rápidas.

8.5.1. Detección de pagos recurrentes

Los umbrales del detector se ajustaron observando una única base de datos, lo cual no demuestra que generalicen. Para comprobarlo se regenera el histórico con 500 semillas distintas y se contrasta el resultado con la verdad conocida por construcción: 0 falsos negativos, 1 falso positivo (0,2 % de los históricos generados), ninguna cadencia mal clasificada y el seguro anual detectado en las 500 semillas.

Conviene señalar el límite de esta validación. El generador asigna a los pagos recurrentes un día fijo de cada mes, mientras que los gastos no recurrentes caen en un día aleatorio. La prueba demuestra por tanto que el detector distingue un cargo de día fijo de uno de día aleatorio, y que no clasifica como recurrente un gasto habitual: una compra semanal, incluso perfectamente regular, no se detecta como suscripción, porque el clasificador solo admite cadencias reconocidas. No cubre, en cambio, el desplazamiento del día de cobro habitual en las domiciliaciones reales, que se cargan el siguiente día hábil. Medido posteriormente, el detector reconoce la cadencia con desplazamientos de hasta dos días, y deja de reconocerla a partir de tres.

8.5.2. Proyección de gasto

El estimador de la proyección a fin de mes se seleccionó comparando tres alternativas sobre los 24 meses completos del histórico, midiendo el error de cada una según el día del mes en que se realiza la estimación.

Método	Día 5	Día 10	Día 15	Día 20
Proporción directa	174,5 %	73,3 %	46,1 %	25,9 %
Pagos fijos por separado	31,7 %	18,4 %	13,6 %	10,6 %
Combinado con media histórica	8,7 %	7,6 %	8,4 %	7,7 %

Cuadro 4: Error medio de cada estimador según el día del mes.

La proporción directa resulta inservible porque el alquiler y los recibos domiciliados se cargan a principios de mes: el día 3, con 890,34 € gastados de los cuales 844,80 € corresponden a pagos fijos, una regla de tres proyectaría 8.903 € para el mes completo.

El estimador adoptado no extrapola los pagos fijos, cuyo coste mensual ya conoce el detector de recurrencia, y combina el ritmo de gasto variable observado con la media histórica, ponderando según lo avanzado que esté el mes. El error se mantiene estable en torno al 8 % durante todo el mes, lo que permite ofrecer una previsión útil desde los primeros días.

Cada proyección se acompaña de su margen de error, calculado aplicando el mismo estimador a cada mes pasado y midiendo la desviación. El margen varía considerablemente según la categoría: 0 % en alquiler o gimnasio, que son pagos fijos conocidos; en torno al 20 % en supermercado o gasolina; y superior al 100 % en los Bizums enviados, que son impredecibles por naturaleza. Ofrecer una cifra única para todas las categorías habría resultado igual de creíble y sustancialmente menos honesto.

8.5.3. Flujo de operaciones

El envío de Bizums se valida mediante 32 casos de prueba, ejecutados con `pytest`, que recorren el flujo completo sin intervención del modelo: reconocimiento de la petición, validación del importe, resolución del contacto, gestión del PIN y atomicidad de la escritura.

Una de las comprobaciones verifica que el bloqueo de escritura se adquiere antes de la primera lectura. Sin esa garantía, dos envíos simultáneos leen el mismo saldo, ambos superan la comprobación y una actualización sobrescribe a la otra: en la reproducción del caso, dos envíos de 600 € sobre un saldo de 1.000 € resultaban ambos aprobados.

8.6. Limitaciones conocidas

La evaluación permite delimitar con precisión lo que el sistema no resuelve:

- **Resolución de referencias entre turnos.** Ante la secuencia “¿cuánto he gastado en supermercado este año?”, “¿y en restaurantes?”, “¿y el año pasado?”, el modelo resuelve la tercera pregunta contra la primera y responde sobre supermercado. Es un comportamiento sistemático, no ocasional.
- **Previsión de meses futuros.** La herramienta proyecta únicamente el mes en curso. Ante una pregunta sobre el mes siguiente, el modelo responde con la cifra del

mes actual sin advertir la diferencia. Se probaron dos correcciones —una regla en el *prompt* y un campo explícito en la respuesta de la herramienta— y ninguna resultó efectiva.

- **Preguntas sin periodo explícito y consultas sobre terceros.** Se documentan como limitaciones asumidas.

Al margen de lo que mide el banco de pruebas, el sistema tiene otros límites que conviene dejar por escrito, porque marcan la distancia entre esta demostración y un producto.

8.6.1. Alcance funcional

- **Un único titular y una única cuenta.** La tabla `cliente` contiene una sola fila y las consultas de saldo se hacen contra `WHERE id = 1`. La tabla `movimientos` no tiene columna de cuenta, así que no hay forma de distinguir entre varias.
- **Catálogo de operaciones reducido.** Se pueden consultar el saldo y los movimientos y enviar un Bizum. No hay transferencias, tarjetas, domiciliaciones ni cancelación de una operación ya realizada.
- **Histórico relativo a la fecha de generación.** Los movimientos se crean a partir del día en que se ejecuta `seed.py`. Si cambia el mes y no se regenera la base de datos, las preguntas sobre “este mes” devuelven resultados vacíos.
- **Solo español.** El *system prompt*, el reconocimiento de voz (`es-ES`) y los formatos de cifra y fecha asumen ese idioma.

8.6.2. Límites técnicos y de operación

- **Memoria de la conversación acotada.** La ventana es de 8192 *tokens* y se conservan 24 mensajes. Una conversación larga pierde el principio, y el historial vive en memoria del proceso: al cerrarse la conexión por inactividad, o al reiniciar el servidor, desaparece.
- **Atención en serie.** Hay un solo proceso y una sola instancia del modelo en la tarjeta gráfica, que no admite dos peticiones a la vez. Con varios usuarios simultáneos las peticiones se encolan y la latencia crece de forma proporcional.
- **SQLite admite un único escritor.** La transacción `BEGIN IMMEDIATE` garantiza la integridad de los envíos, pero lo hace serializándolos.
- **Las latencias dependen del equipo.** Las cifras de esta memoria se midieron sobre un portátil concreto y conectado a la corriente; en otro equipo, o con la batería limitando la tarjeta gráfica, cambian de forma apreciable.
- **La interfaz necesita conexión.** El modelo y los datos son locales, pero el navegador descarga de la red la biblioteca de gráficos (Vega-Lite), las tipografías y los iconos de los comercios. Sin internet el asistente responde y habla igual, pero los gráficos no llegan a dibujarse.

8.6.3. Límites de la demostración en materia de seguridad

- **El acceso no es autenticación real.** La pantalla de inicio se valida en el navegador y el punto de entrada `/ws` no comprueba ninguna credencial: quien conozca la dirección puede abrir una conversación.
- **El PIN está fijado en el código** (`config.py`), sin cifrado ni almacenamiento seguro. Su papel aquí es demostrar que la clave nunca llega al modelo, no proteger un activo real.
- **No hay registro de auditoría.** Las operaciones modifican el saldo pero no dejan traza de quién las pidió ni desde dónde.

8.6.4. Límites de la propia evaluación

- **El resultado varía entre repeticiones** en uno o dos casos aunque no cambie nada del sistema. Por eso las cifras se obtienen siempre sobre tres repeticiones, y una diferencia de un caso no se interpreta como mejora ni como regresión.
- **El banco tiene 44 preguntas.** Cubre los tipos de consulta del enunciado y los añadidos durante el desarrollo, pero no es exhaustivo: hay formas de preguntar que no están representadas.
- **No se prueba la interfaz de forma automática.** Las pruebas llegan hasta los eventos que emite el servidor; lo que ocurre después en el navegador se ha comprobado a mano.

Delimitar estas limitaciones con precisión resulta más útil que un resultado global elevado, ya que indica en qué condiciones conviene confiar en el sistema y en cuáles no.

9. Conclusiones y Cumplimiento del Baremo

9.1. Conclusiones

El desarrollo de “Habla con tu dinero” ha permitido implementar un asistente conversacional capaz de consultar información financiera, realizar determinadas operaciones y generar representaciones visuales a partir de las peticiones del usuario.

Uno de los principales aspectos de la arquitectura es la separación de responsabilidades entre el modelo de lenguaje y el código de la aplicación. El LLM se utiliza principalmente para interpretar las peticiones, generar consultas SQL, seleccionar herramientas y definir la estructura de las visualizaciones. Por otro lado, el *backend* se encarga de realizar los cálculos, consultar los datos y ejecutar las operaciones que modifican el estado de la aplicación.

Esta separación también se aplica a las operaciones que requieren un mayor control, como el envío de Bizums. La validación de los datos, la autenticación mediante PIN y la actualización del saldo se realizan fuera del modelo de lenguaje y mediante la lógica del *backend* y las transacciones de la base de datos.

Durante el desarrollo también se han realizado diferentes ajustes para reducir el tiempo de respuesta. Un ejemplo es la resolución directa de determinadas consultas, como la del saldo, que se resuelven sin recurrir al modelo.

Otro es la posición del motor visual dentro del turno. La generación de la tabla o el gráfico no se ejecuta en paralelo con la respuesta, sino después del evento `fin_respuesta`, cuando el texto ya se ha emitido y la síntesis de voz, que arranca con la primera frase, está en marcha. De este modo la generación queda fuera del camino crítico: el usuario escucha la respuesta de inmediato y el elemento visual aparece mientras la está oyendo, acompañado de un indicador de que se está preparando. En las pruebas realizadas, este cambio redujo el tiempo hasta la respuesta hablada de 12,4 a 6,7 segundos en las preguntas que generan un elemento visual.

En conjunto, el proyecto muestra que es posible utilizar un modelo de lenguaje como parte de una aplicación de gestión financiera manteniendo fuera del modelo las operaciones que requieren cálculos, validaciones y modificaciones de los datos. Esta organización permite aprovechar el procesamiento de lenguaje natural sin depender del modelo para las partes críticas de la aplicación.

9.2. Cumplimiento del baremo técnico del reto

El desarrollo se ha realizado tomando como referencia los criterios de evaluación establecidos para el reto. Las principales funcionalidades implementadas son las siguientes:

- **Text-to-SQL:** se ha implementado la traducción de consultas en lenguaje natural a sentencias SQL mediante la herramienta `consultar_movimientos`. El esquema de la base de datos se incorpora al *System Prompt* y el sistema permite corregir determinadas consultas cuando se produce un error durante su ejecución. Las consultas sobre el histórico se realizan mediante una conexión SQLite de solo lectura (`mode=ro`) y se aplican validaciones adicionales antes de ejecutarlas.
- **Operaciones Bancarias:** el asistente permite realizar operaciones como la consulta del saldo y el envío de dinero mediante Bizum. El flujo de Bizum incluye la extracción y validación del importe, la resolución del destinatario, la introducción del PIN desde el frontend y la ejecución de la operación mediante una transacción SQLite iniciada con `BEGIN IMMEDIATE`.
- **Gráficos sin plantillas:** las visualizaciones se generan a partir de los resultados concretos de cada consulta, sin utilizar una plantilla fija para cada caso. El modelo genera especificaciones compatibles con Vega-Lite v5 y determina elementos como el tipo de gráfico, los ejes y los títulos.
- **Lógica visual:** el backend analiza inicialmente si los resultados obtenidos son adecuados para una representación visual. Cuando lo son, el modelo determina si la información debe mostrarse como tabla o como gráfico y genera la estructura correspondiente. La decisión incluye el campo `razonamiento`, utilizado para indicar el motivo de la elección.
- **Analítica avanzada:** se han implementado funcionalidades adicionales para analizar los movimientos del usuario, como la detección de pagos recurrentes y la proyección del gasto mensual. Estos cálculos se realizan en Python mediante reglas y métodos estadísticos, mientras que el modelo de lenguaje se utiliza para interpretar la consulta y presentar los resultados.

10. Líneas de trabajo futuras

Las limitaciones descritas en la sección anterior no son todas del mismo tipo: unas se resuelven con trabajo acotado y otras exigirían rediseñar una parte del sistema. Se recogen aquí ordenadas por esa distancia, indicando en cada caso qué limitación concreta vendrían a resolver.

10.1. Mejoras sobre lo ya construido

- **Reescritura de la pregunta con el contexto.** Es la respuesta al fallo de resolución de referencias entre turnos. Antes de llamar al modelo principal, una llamada corta reformularía “¿y el año pasado?” como una pregunta completa y autónoma a partir de los últimos turnos. Tiene un coste claro —una llamada más por turno— que habría que medir antes de adoptarlo, ya que la latencia es una prioridad del proyecto.
- **Cambiar a un modelo más capaz si el equipo lo permite.** El modelo es una variable de entorno (`MODELO`, con `qwen3:8b` por defecto), así que sustituirlo no supone tocar el código. Lo que decide es el hardware: un modelo mayor necesita más memoria en la tarjeta gráfica y genera más despacio, y en este proyecto la latencia es una prioridad. Es, además, la otra vía para atacar el fallo de referencias entre turnos, que es una limitación de razonamiento propia de un modelo de 8.000 millones de parámetros más que un problema de diseño. El banco de pruebas es justamente la herramienta para decidirlo: permite comparar precisión y latencia entre dos modelos sobre las mismas 44 preguntas y comprobar si la mejora compensa el tiempo de respuesta añadido.
- **Previsión de meses distintos del actual.** La herramienta `proyectar_gasto` calcula siempre sobre el mes en curso. Añadirle un parámetro de mes objetivo, y que rechace explícitamente los meses para los que no hay base suficiente, evitaría que el modelo presente la cifra del mes actual como si fuera la del siguiente.
- **Servir las dependencias del navegador en local.** Descargar Vega-Lite, las tipografías y los iconos y servirlos desde `frontend/` haría que la aplicación funcionase sin conexión y eliminaría la única dependencia externa que queda.
- **Registro de auditoría.** Guardar en una tabla cada consulta SQL generada, cada herramienta invocada y cada operación ejecutada, con su resultado y su marca de tiempo. Es poco trabajo y encaja con el enfoque de IA responsable: permite reconstruir después por qué el asistente respondió lo que respondió.
- **Ampliar el banco de pruebas y automatizarlo.** Las preguntas actuales cubren los tipos del enunciado, pero el banco se ejecuta a mano. Lanzarlo de forma automática ante cada cambio del *prompt* detectaría las regresiones en el momento en que se introducen, que es justo el problema que motivó construirlo.
- **Lectura de recibos por OCR.** Existe un prototipo en una rama aparte que permite adjuntar la imagen de una factura: el *backend* extrae el texto con Tesseract, en local, y el modelo identifica el comercio, el importe y la categoría. Se decidió no integrarlo por la gran variedad de formatos de recibos y facturas posibles, que

hace difícil garantizar una extracción fiable. Además, el texto completo del recibo entraría en el historial y ocuparía unos 600 *tokens* de la ventana de contexto, aunque esto se resolvería conservando solo los datos extraídos. Una base de datos semántica de documentos solo compensaría si el usuario fuera a consultar muchas facturas a la vez. Luego consideramos que para una demo no era necesario.

10.2. Mejoras que requieren rediseño

- **Autenticación real y gestión de sesión.** Sustituir la pantalla de acceso por una validación en el servidor, proteger el punto de entrada */ws* con un testigo de sesión, almacenar el PIN cifrado y limitar el número de intentos por origen. Es el paso obligado para que la aplicación deje de ser una demostración.
- **Soporte multiusuario.** Implica añadir la noción de cuenta a las tablas y a todas las consultas, asociar cada conexión a un titular y sustituir SQLite por un gestor que admita varios escritores. El patrón de validación, PIN y transacción se mantendría; lo que cambia es de dónde sale la cuenta sobre la que se opera.
- **Escalado del modelo.** Con un único modelo en una tarjeta gráfica, las peticiones se atienden una detrás de otra. Atender a varios usuarios a la vez exigiría una cola explícita y varias réplicas del modelo, además de decidir qué ocurre cuando la espera supera lo tolerable.
- **Ampliar el catálogo de operaciones.** Transferencias, domiciliaciones o gestión de tarjetas encajarían en el mismo esquema que el Bizum: extracción de la intención, validación en el *backend*, confirmación con PIN y transacción. Cada nueva operación añade, eso sí, texto al *system prompt*, y ya se midió que el tamaño del *prompt* degrada las reglas existentes.
- **Persistencia de la conversación.** Guardar el historial fuera del proceso permitiría retomar una conversación tras una desconexión y abriría la puerta a que el asistente recuerde preferencias entre sesiones.

De todas ellas, las dos que más cambiarían la experiencia son la reescritura de la pregunta con el contexto, porque elimina el único fallo sistemático que la evaluación detecta, y la autenticación real, porque es la diferencia entre una demostración y algo que pueda tocar dinero de verdad.

11. Reparto del trabajo

El desarrollo se ha organizado de forma que cada integrante asumiera la responsabilidad principal de un conjunto de áreas del proyecto. La Tabla 5 lo resume y los apartados siguientes lo detallan.

11.1. Pablo Martín Palomino

- Base inicial del proyecto: servidor FastAPI con WebSocket, ejecución de consultas SQL en modo solo lectura, primeros gráficos con Vega-Lite y voz en el navegador.

Integrante	Áreas principales
Pablo Martín Palomino	Base del agente y modelo local, motor visual, analítica avanzada, evaluación y robustez del flujo de Bizum
Natalia Yue Gallardo Pretel	Flujo conversacional de Bizum, seguridad y experiencia en la interfaz, y redacción de la memoria
Pablo Reyes Sousa	Interacción por voz y personalización de la voz del asistente

Cuadro 5: Resumen del reparto del trabajo.

- Planificación del trabajo y migración a un modelo local (**qwen3:8b** sobre Ollama), incluido el script de arranque que amplía el contexto a 8192 tokens.
- Detección de pagos recurrentes y control del tamaño del contexto del modelo.
- Proyección del gasto a fin de mes, con el margen de error de cada categoría medido sobre el histórico.
- Motor visual: llamada dedicada para generar la especificación, decisión determinista en el backend sobre cuándo mostrar un elemento visual y tablas como segunda representación.
- Banco de evaluación de 44 preguntas y pruebas automáticas del flujo de Bizum y del detector de recurrencia.
- Robustez del flujo de Bizum: validación del importe antes de pedir el PIN, unificación de las dos vías de entrada, transacción **BEGIN IMMEDIATE** y acceso a la base de datos fuera del bucle de eventos.
- Reducción de la latencia: *streaming* por frases, recorte del *system prompt* y generación visual posterior a la respuesta hablada.
- Documentación del repositorio en español e inglés.

11.2. Natalia Yue Gallardo Pretel

- Atajo para consultar el saldo sin llamar al modelo y reducción del número de tokens y llamadas por respuesta.
- Flujo conversacional de Bizum: validación del contacto, herramienta de contactos, importe opcional y normalización de las respuestas afirmativas y negativas en la confirmación.
- Primera versión del historial de conversación y corrección de las consultas de gasto, incluida la eliminación del formato Markdown en las respuestas.
- Capa de seguridad en la interfaz: pantalla de acceso, teclado numérico para el PIN y aviso de los intentos restantes.

- Experiencia de usuario: justificante de Bizum con concepto, personalización del perfil, del color y del tamaño de letra, y logotipos de los comercios en las tablas.
- Extracción de información financiera mediante imágenes aportadas por el usuario.
- Estructura de la memoria en $\text{\LaTeX}$, portada y redacción de los capítulos.

11.3. Pablo Reyes Sousa

- Mejora de la interacción por voz: activación del micrófono con la barra espaciadora e indicador visual de que la lectura en voz alta está activada.
- Panel de personalización de la voz del asistente: elección entre las voces disponibles en español, ajuste del tono y de la velocidad, y prueba de la configuración antes de guardarla.

11.4. Uso de asistentes de IA

Durante el desarrollo se han utilizado asistentes de programación basados en inteligencia artificial como apoyo en la escritura y revisión de código, la depuración de errores, la redacción de documentación y la generación del catálogo de comercios de los datos ficticios. Su uso ha estado siempre sujeto a la revisión del equipo.